

## AULA PRÁTICA N.º 10

### Objectivos:

- A norma IEEE 754. Representação de números reais (tipos *float* e *double*).
- Programação com a unidade de vírgula flutuante do MIPS. Parte 2.

### Guião:

1. A função seguinte calcula o valor de  $x^y$ , com "x" real e "y" inteiro (positivo ou negativo).

```
float xtoy(float x, int y)
{
    int i;
    float result;

    for(i=0, result=1.0; i < abs(y); i++)
    {
        if(y > 0)
            result *= x;
        else
            result /= x;
    }
    return result;
}

int abs(int val)
{
    if(val < 0)
        val = -val;
    return val;
}
```

*Handwritten notes:*

- $x$  is highlighted in yellow,  $y$  in purple. Arrows point to  $\$a0$  (for  $x$ ) and  $\$f12$  (for  $y$ ).
- $i$  is highlighted in blue.
- $result$  is highlighted in green.
- Red note: "(se calcular  $abs(y)$  só dps de  $i=0$  e  $result=1.0$  ent  $i$  e  $result$  têm de ser calculados)
- Red note: "se os f:  $\$f20 \rightarrow \$f30$ "
- Red note: "val = 0 - val" next to  $val = -val$ .
- Red note: " $\$f0$ " and " $\$a0$ " with arrows pointing to the return statement.
- Red note: " $\$v0$ " with an arrow pointing to the return statement of `abs`.

- a) Escreva, em linguagem C, a função `main()` para teste da função `xtoy()`.
  - b) Traduza as duas funções para *assembly* do MIPS e teste o conjunto.
2. A função seguinte implementa um algoritmo de cálculo da raiz quadrada (conhecido como "Babylonian method").

```
double sqrt(double val)
{
    double aux, xn = 1.0;
    int i = 0;
    if(val > 0.0)
    {
        do
        {
            aux = xn;
            xn = 0.5 * (xn + val/xn);
        } while((aux != xn) && (++i < 25));
    } else
        xn = 0.0;
    return xn;
}
```

*Handwritten notes:*

- $xn$  is highlighted in red.
- Red note: " $\$f12$ " with an arrow pointing to  $xn$ .
- Red note: " $\$t0$ " with an arrow pointing to  $i$ .
- Red note: " $\$f0$ " with an arrow pointing to the return statement.

- a) Escreva, em linguagem C, a função `main()` para teste da função `sqrt()`.
- b) Traduza as duas funções para *assembly* do MIPS e teste o conjunto.

3. As duas funções seguintes calculam a variância e o desvio padrão dos elementos de um array.

```
double var(double *array, int nval)
{
    int i;
    float media, soma;

    media = (float)average(array, nval);

    for(i=0, soma=0.0; i < nval; i++)
        soma += xtoy((float)array[i] - media, 2);

    return (double)soma / (double)nval;
}

double stdev(double *array, int nval)
{
    return sqrt( var(array, nval) );
}

#define SIZE 10
int main(void)
{
    double arr[SIZE];
    int i;
    for(i=0; i < SIZE; i++)
        arr[i] = read_double();

    print_double( average(arr, SIZE) );
    print_double( var(arr, SIZE) );
    print_double( stdev(arr, SIZE) );

    return 0;
}
```

$$var = \frac{1}{N} \sum_{i=1}^N \left( V_i - \overline{V} \right)^2$$

dispersão de valores em relação à média

$$rms = \sqrt{var}$$

Traduza as funções anteriores para *assembly* do MIPS e teste o seu funcionamento.

## Exercícios adicionais

- a) Recupere a função **main()** que escreveu na aula anterior (por exemplo no exercício 4b). Rescreva a função de modo a que efetue o seguinte conjunto de operações:
- Leia da consola 11 valores de temperatura em graus Fahrenheit em formato inteiro.
  - Após a leitura de cada valor, converta a temperatura de graus Fahrenheit para graus Celsius e armazene o resultado num *array* de reais, precisão dupla.
  - Seguidamente, o programa deverá mostrar:
    - O *array* de temperaturas completo.
    - A temperatura máxima.
    - A temperatura média e a mediana.
    - A variância da amostra.
    - O desvio padrão da amostra.
- b) Converta a função **main()** para *assembly* do MIPS e teste o programa completo (terá que utilizar as funções já implementadas na aula anterior).